

structlog — Structured Logging

Why the node logs machine-readable key-value events instead of prose sentences, and how `structlog` makes a long-running daemon observable.

SUBJECT hathor-core · Part I · Track C (the stack)

CHAPTER 17 · Foundations · The Stack

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Logging · structlog · Structured events · Key-value pairs · Log levels · Processors · Context binding · JSON logs · Observability

Table of Contents

Chapter 17 — structlog: Structured Logging	3
17.1 The problem: print doesn't scale to a daemon	3
17.2 Structured logging: events, not sentences	4
17.3 The structlog model: loggers and the processor pipeline	5
17.4 Why structlog, and not the alternatives?	5
17.5 Bridge — logging across the project	6
Recap	7

Chapter 17 — structlog: Structured Logging

What you'll learn in this chapter

- Why `print()` is not how production software records what it does, and the problem **logging** solves.
- What makes logging structured — **key-value events** instead of formatted sentences — and why that matters for a daemon you can't watch live.
- The core model of **structlog**: loggers, **log levels**, bound **context**, and the **processor** pipeline that formats output (human-readable in dev, JSON in production).
- How `hathor-core` configures and uses it (`get_logger`, `logger.info(event, **fields)`).
- **Why structlog** over `print` and Python's standard `logging` — the trade-off.

A node runs unattended for months on a remote server. When something goes wrong — a peer misbehaves, a transaction is rejected, sync stalls — there is no one watching the screen. The only record of what happened is what the node wrote to its **logs**¹. So how a node logs is not a detail; it is the difference between a diagnosable incident and a mystery. `hathor-core` uses **structlog**², a library for **structured logging**, and this chapter explains both the practice and the tool.

It follows the §5 primer shape, ending with the "why this, not the alternatives" discussion.

17.1 The problem: `print` doesn't scale to a daemon

When learning, you debug with `print()`. It fails as production infrastructure for concrete reasons:

- **No severity.** Everything is the same "level." You can't say "this is routine info" vs. "this is a warning" vs. "this is a fatal error," and you can't filter to just the serious ones when the log is millions of lines.
- **No destination control.** `print` goes to standard output. Real software needs to route logs to files, to the system journal, to an aggregation service — and to switch verbosity without editing code.

- **No structure.** A printed sentence like `peer 0xabc disconnected after 30s` is for a human reading one line. When you have millions of lines and need to ask "how many disconnections happened, and which peers?", prose is nearly unsearchable — you're reduced to fragile text-matching.

Logging addresses the first two: a logging framework gives **levels** (DEBUG/INFO/WARNING/ERROR/CRITICAL³) and configurable routing. Structured logging addresses the third, and it is the idea worth dwelling on.

17.2 Structured logging: events, not sentences

The shift is from logging formatted prose to logging data. Instead of building a sentence:

```
logger.info(f"peer {peer_id} disconnected after {seconds}s") # a string
```

you log an **event name plus key-value fields**:

```
logger.info("peer disconnected", peer_id=peer_id, seconds=seconds) # structured
```

The difference looks small and is profound. The structured call produces a record with fields — `event="peer disconnected" peer_id="0xabc" seconds=30` — which can be rendered as JSON⁴ and then **queried like a database**: "show every `peer disconnected` event where `seconds > 60`," "count events grouped by `peer_id`." The prose version can only be grepped with brittle regexes. For a system whose health is understood after the fact from its logs — the definition of **observability**⁵ — this queryability is the whole game.

A second benefit: **context binding**. structlog lets you bind fields to a logger once and have them attached to every subsequent event automatically. Bind `peer_id` when a connection opens, and every log line for that connection carries it without your repeating it — so you can later filter the entire life of one connection out of the interleaved firehose of thousands.

PROSE LOG (grep-only)	STRUCTURED LOG (queryable)
"peer 0xabc disconnected after 30s"	{event: "peer disconnected",
"block accepted height 42"	peer_id: "0xabc", seconds: 30,
→ text matching, fragile	conn_id: 7}
	→ filter/group/count by field

17.3 The structlog model: loggers and the processor pipeline

structlog's design has two parts you'll see in the code.

Getting a logger. A module obtains a logger with `get_logger()` and logs by calling a level method with an event string and fields:

```
from structlog import get_logger
logger = get_logger()
logger.info("removing all events and related data...") # event, optionally + fields
```

This pattern recurs across `hathor-core` (e.g. `hathor_cli/reset_event_queue.py`). The first positional argument is the **event** (a short, stable name); keyword arguments are the structured fields.

The processor pipeline. The mechanism that makes structlog flexible is a chain of **processors**⁶: each log event passes through an ordered list of small functions, each transforming the event dict, before it is finally rendered. Processors add a timestamp, add the level, add bound context, and — at the end — a **renderer** turns the event dict into output. This is the decorator/pipeline idea from Chapter 4 applied to log records: behavior composed as a sequence of transformations.

The renderer at the end of the chain is what makes one log style for humans and another for machines from the same logging calls:

- In **development**, a console renderer produces colorized, aligned, human-friendly lines. `hathor-core` even subclasses structlog's `ConsoleRenderer` (a custom `ConsoleRenderer` in `hathor_cli/util.py`) to tune that output.
- In **production**, a JSON renderer emits one JSON object per line — exactly the queryable form of §17.2, ready for a log-aggregation system.

The configuration that assembles this pipeline lives in `hathor_cli/util.py` (the `setup_logging` function calls `structlog.configure(...)`, wiring the processor chain and renderer; it also optionally adds a Sentry⁷ processor for error reporting). The key idea: the same `logger.info(...)` calls throughout the codebase produce friendly output locally and machine-parseable JSON in production, because only the final renderer changed.

17.4 Why structlog, and not the alternatives?

vs. `print`. Covered in §17.1: no levels, no routing, no structure. Fine for a script, unfit for a daemon.

vs. Python's standard logging. The standard library does have levels and routing (handlers, formatters), and it's the baseline. What it lacks natively is structure and context binding: standard logging is oriented around formatting a message string, and attaching arbitrary key-value fields (and especially binding them across many calls) is awkward. structlog is built around the event-dict-and-fields model from the ground up, and crucially it **integrates with**, rather than replaces, standard logging — it can route its output through the stdlib's handlers, so you keep the mature routing and gain the structure. That "best of both" is the deciding factor.

vs. other structured loggers (loguru, etc.). loguru is a popular ergonomic alternative; the choice between it and structlog is largely taste and ecosystem. structlog's processor-pipeline model and its clean stdlib integration suit a large application that wants explicit control over how events are built and rendered.

The honest summary: structlog costs a little setup (the processor pipeline must be configured once) and a small mental shift (log events with fields, not sentences). In return a long-running node produces logs you can query, filter, and aggregate — the foundation of diagnosing anything in production. For unattended infrastructure, that is not optional polish; it's how the system is operated.

17.5 Bridge — logging across the project

Bridge — structlog in the project and the stack:

- **Configured once at startup.** `setup_logging` in `hathor_cli/util.py` builds the processor pipeline and picks the renderer (console vs. JSON) — called during the boot sequence of **Chapter 21**.
- **Used everywhere.** `get_logger()` + `logger.info(event, **fields)` appears across `hathor/` — you'll see it in nearly every Part II chapter.
- **The processor pipeline is Chapter 4's idea.** A chain of transforming functions ending in a renderer — decorators/dispatch applied to log records — **Chapter 4**.
- **It feeds observability alongside metrics.** Logs answer "what happened"; Prometheus metrics answer "how much/how often" — **Chapter 42**.
- **Optional Sentry integration.** Errors can be forwarded to Sentry via a processor — an example of the optional-dependency fallback pattern of **Chapter 4**.

Recap

Concept	What it is	Why it matters
logging	recording what software does, with levels + routing	the daemon's only witness
log level	severity (DEBUG...CRITICAL)	filter signal from noise
structured logging	events with key-value fields, not prose	logs become queryable
context binding	fields attached once, on every later event	trace one connection's whole life
processor pipeline	chain of transforms ending in a renderer	same calls → console or JSON
renderer	final formatter of the event	human-readable dev / JSON prod
<code>get_logger</code> / <code>.info(event, **f)</code>	the usage pattern	throughout <code>hathor-core</code>
why structlog	structure + binding + stdlib integration	vs print / bare logging

structlog gives `hathor-core` structured logging: every log call records a named event with key-value fields rather than a prose sentence, so the logs a long-running node leaves behind can be filtered, grouped, and counted like data instead of grepped like text. Its processor pipeline lets the same logging calls render as friendly console output in development and machine-parseable JSON in production, and context binding lets a single connection or transaction be traced out of the interleaved stream. It is chosen over `print` (no levels/structure) and bare stdlib logging (awkward structure) while integrating with the latter's routing. The next chapter turns from recording what the node does to validating what enters it: **Pydantic**, which checks external data at the boundary using the very type-hint syntax of Chapter 5.

-
1. Logging is the practice of emitting timestamped records of a program's activity to a destination (console, file, service), typically with severity levels and configurable routing, so behavior can be understood after the fact. ↵
 2. structlog is a Python library for structured logging. It models each log entry as an event name plus a dictionary of key-value fields, passes it through a pipeline of processors, and renders it (human-readable or JSON). ↵

3. A log level is a severity tag on a log entry — commonly DEBUG, INFO, WARNING, ERROR, CRITICAL — used to filter which entries are recorded or shown, so routine detail can be separated from serious problems. ↩
4. JSON (JavaScript Object Notation) is a text format for structured data as key-value objects and arrays. Emitting logs as one JSON object per line makes them directly ingestible by log-analysis tools. ↩
5. Observability is the degree to which a system's internal state can be understood from its external outputs (logs, metrics, traces). Structured logs are a pillar of it, especially for systems no one watches live. ↩
6. A processor in structlog is a function in the pipeline that receives and transforms a log event dict (adding a timestamp, level, bound context, etc.) before the final renderer turns it into output. ↩
7. Sentry is an error-tracking service that aggregates and alerts on exceptions from running software. structlog can forward error events to it via a processor; it is an optional dependency in `hathor-core`. ↩